Secure Document Storage Application

Proposal

Artem Surzenko, Danyil Tymchyk and Illia Stefanovskyi

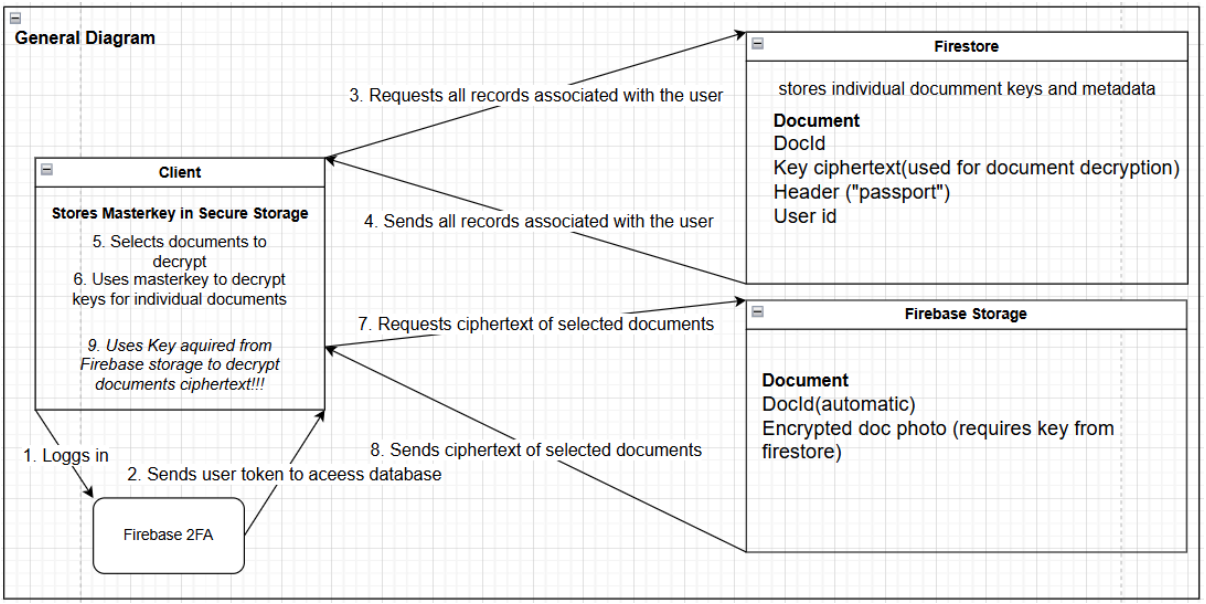
26/12/2025

Introduction

Nowadays everyone needs to be able to provide documents in digital format on regular basis, this leads to many cases of people storing photos of those in plain format on their devices or other storage means that are not supposed to be used for sensitive data. Our goal is to deliver an Android application that will enable secure storage of encrypted documents in a cloud service enhancing zero knowledge principle. Layers of security will include authentication and single device access point.

Description and Architecture

We will integrate multiple firebase services into our project, those will be used to allow secure authentication and storage of ciphertext ensuring that even in the event of data exposure documents remain secure. Android KeyStore on the user's device will be used to store a masterkey for decryption of individual document keys and firebase authentication will serve for database access. Modern algorithms will be used for encryption of both documents and keys on the client side. A strong random masterkey will be generated during sign up. Authentication will require a password and validated email with an option to change these in the future.



Scope

Within Scope

- Account creation
- Authentication with email validation
- Masterkey storage
- Individual document keys storage
- Encrypted documents storage
- Single device access per account
- CRUD operations on documents
- User credentials modification
- Encryption/decryption of documents and keys

Out of Scope

- Account transition from one device to another
- Document verification
- Integrity checks (hashing)
- Document sharing
- Data backup

Functional Requirements

- Sign up
- Sign in
- Unique masterkey generation
- Individual document key generation on client side
- Individual document key encryption on client side
- Document encryption on client side
- Individual document key decryption on client side
- Document decryption on client side
- CRUD operations on documents
- Single device per account
- User credentials modification (email and password)

Risks and Mitigation

Risk 1: Encryption Implementation Errors

Problem: Incorrect use of AES-256-GCM, weak IV generation, or key derivation flaws.

Mitigation:

- Use AndroidX Security Crypto or Tink libraries
- IV generated with SecureRandom (unique per encryption)
- Code review and security testing before launch
- Follow OWASP Mobile Security guidelines

Risk 2: MasterKey Compromise

Problem: If masterKey is exposed on device, all documents become decryptable.

Mitigation:

- MasterKey stored in Android KeyStore (hardware-backed when available)
- Non-exportable and non-extractable
- Protected by device lock/biometric
- Clear sensitive data from memory after use
- Automatic session timeout

Risk 3: Unauthorized Device Access

Problem: Different devices could access the same account if credentials are stolen.

Mitigation:

- Single-device-per-account policy enforced
- Device ID binding in Users collection
- Device fingerprinting (optional enhancement)

Risk 4: Weak Authentication

Problem: Stolen credentials could grant full account access.

Mitigation:

- Firebase Authentication with strong password requirements
- Two-Factor Authentication (email-based, 10-min expiry)
- Rate limiting on login attempts (max 3 attempts per 5 min)
- Session timeout after 30 minutes inactivity

Risk 5: Data Exposure During Transmission

Problem: Documents could be intercepted during upload/download if not properly encrypted.

Mitigation:

- All documents are encrypted client-side BEFORE upload
- Use HTTPS for all Firebase communications
- Verify SSL/TLS handshake(optional)

Conclusion

Project presents a secure document storage Android application that applies encryption and decryption operations on the client side. Firebase services are used for authentication and encrypted data storage, while sensitive keys are protected using Android Keystore. The defined scope and architecture ensure that the system remains secure, scalable, and feasible.